

CS482 Assignment 4 - Readme

Name: Yuanhao Liu (Jim Liu)

Student ID: 20775207

Date: 2026-03-31

=== How to Run ===

file: assn4.py (or assn3.py)

run:

```
python assn4.py  
[type input string and press Enter]
```

or via pipeline:

```
echo banana | python assn4.py
```

=== Algorithm Overview ===

Build suffix array of input string using prefix doubling + radix sort.
Achieves $O(n \log n)$ time and $O(n)$ space.

Core idea: sort suffixes by substrings of length $L = 1, 2, 4, \dots$
Each round uses ranks from previous round as sort keys, so only two integers per suffix are compared instead of full substrings.
Doubling continues until all suffix ranks are distinct or $L \geq n$.

=== What Each Part Does ===

1. `singleFreqSort(idxArr, keyList, keyRange)`

- counting sort on suffix index array
- key values precomputed in `keyList`, range $0..keyRange$
- count freq $\rightarrow$ prefix sum $\rightarrow$ forward fill (keeps stable order)

2. `dualFreqSort(idxArr, rankArr, L, curLen)`

- two-pass radix sort on $(rankArr[i], rankArr[i+L])$
- pass 1: sort by second component $rankArr[i+L]$
out-of-bound index $(i+L \geq curLen) \rightarrow$ key = 0 (sentinel rank)
- pass 2: first component $rankArr[i]$
- stable property from `singleFreqSort`, pair order carries over

3. `build_suffix_array(s)`

- append sentinel '\$' (lexicographically smallest) to input
- $L=1$ init: map chars to keys ('\$' $\rightarrow 0$, 'a' $\rightarrow 1$..'z' $\rightarrow 26$),
run singleFreqSort, assign initial ranks by comparing adjacent chars
- main loop: call dualFreqSort $\rightarrow$ renaming $\rightarrow L *= 2$
renaming writes to newRankArr (not in-place)
terminates when $\text{rankMax} == \text{curLen}-1$ (all distinct) or $L \geq \text{curLen}$
- return sorted suffix index array

=== Renaming (Equivalence Classes) ===

After each sort pass, suffixes get new rank numbers based on their current sorted order. Same pair as previous $\rightarrow$ same rank, different $\rightarrow$ rank+1. This converts sort results into integer keys for the next round, so each doubling step only needs to compare two integers per suffix instead of reading the original string characters.

=== Time Complexity ===

$O(n \log n)$ where $n = \text{len}(s) + 1$ (including sentinel)

- outer loop: $O(\log n)$ rounds, L doubles each round
- each round: two singleFreqSort calls + renaming pass $\rightarrow O(n)$
- per round $O(n)$, $O(\log n)$ rounds total $\rightarrow O(n \log n)$

=== Space Complexity ===

$O(n)$

- all arrays (idxArr, rankArr, charKeyList etc.): $O(n)$
- no suffix string copies, index only

=== Edge Cases ===

- all-same characters (e.g. "aaaaaa"):
ranks stay tied for multiple rounds, loop runs to $L \geq \text{curLen}$
terminates via second condition, not early exit
- length-1 input:
after appending '\$', $\text{curLen}=2$, one doubling round covers it
- sentinel out-of-bound access:
 $\text{rankArr}[i+L]$ where $i+L \geq \text{curLen} \rightarrow$ fill key=0
(sentinel '\$' rank stays 0 throughout all rounds)